A Cache in the Weights? Online Learning and Temporal Locality in Small Language Models

Preliminary results

Ruben Tous
Affiliation

Draft, November 6, 2026

Abstract

Attention forgets everything beyond its window, but the text a language model reads stays on the same topic, characters and vocabulary for far longer. We ask whether a small language model that keeps learning online, one SGD step on each chunk after predicting it, uses part of its weights as a cache of its current context. Pythia-14M reads PG-19 books in independent chunks of 256 tokens. Online learning lowers its loss by 0.099 ± 0.006 nats/token, and by 0.301 for rare tokens last seen in an earlier chunk under 1k tokens back. This long-range gain is local (only 18% of it survives shuffling the chunks), tied mostly to the current book rather than to the recency of single tokens, more than a count cache beyond 1k tokens, and stored in the MLPs below the output layer. Probing frozen snapshots of the learner shows that the memory of a book is evicted quickly (its gain on the book's second half halves within about 5k tokens of other books) and that general knowledge is kept within the domain but not outside it: on unseen PG-19 books the learner loses nothing, on WikiText it loses 0.196 ± 0.006 nats/token, about twice its gain on the stream. Making the cache more aggressive (higher learning rates, a single layer, decay) gains more on recent material but loses much more on everything else, at every level of locality we tested. A small decaying fast part on one middle MLP, added on top of the slow learner, gives a small but robust extra gain with strong locality ($+0.012 \pm 0.001$ on books read in order, $+0.021 \pm 0.003$ on long books) and costs little without it (-0.008); meta-learning per-tensor rates and decays learns almost nothing beyond its starting values. Over 200k-token books, online learning closes about half of the gap between Pythia-14M and a frozen Pythia-31M, but Pythia-31M learning online gains just as much: at these sizes, online learning does not favor the smaller model. Plain SGD barely moves Pythia-70M's MLPs; a per-tensor normalized update more than doubles its gain and gives it a clear but much smaller cache than at 14M and 31M, and with every update rule tuned the same way the smaller models do not pay more general knowledge per nat of gain. These are preliminary results with one seed per size, three sizes and few books.

1 Introduction

What a model reads at inference time has strong temporal locality. A book keeps its characters, places and turns of phrase for hundreds of thousands of tokens; a conversation, a code base or a document collection behaves the same way. Computer memory hierarchies have long exploited this property with a small cache that holds the recently used data [5]. A transformer language model has such a cache in its attention window, but the window is short: anything older is forgotten.

Weights updated online could keep a memory of the current context beyond the window. Dynamic evaluation [12, 13, 19] and recent test-time training methods [6, 2, 23] do this and report lower perplexity or better long-context performance. What is less clear is what kind of memory the updated weights hold. If online learning turns part of a model into a cache, the gain should be largest for material seen recently and fall with distance (recency), new material should push old material out (eviction, the useful side of the catastrophic interference that sequential training causes [16]), and the general knowledge in the weights should survive. Alternatively, the gain may come mostly from adapting to the domain or style of the text in general, which needs no memory of the recent past. A cache would also matter most where capacity is scarce: a small model could rely on it more than a larger one.

We study these questions with a deliberately simple setup and small models, where controls are cheap. Pythia models [4] read PG-19 test books [18] in chunks of 256 tokens, each scored with no context from the previous

chunk, so anything that helps beyond 256 tokens has to come through the weights; after scoring a chunk, the model takes one SGD step on it. Our contributions:

- **Controls that split the online gain**: by distance to the token's previous occurrence, into a local part (which needs the order of the text) and a global part (a shuffled stream), into same-book and earlier-book memory, and against a count cache. The long-range gain is local and mostly tied to the current book.
- The **location of the memory**: updating only the MLPs, or one middle layer's MLP, keeps most of the long-range gain; the output layer alone does not.
- **Locality as a variable**: streams that interleave K books, which show that making the cache more aggressive does not pay off at any locality level tested, while a small decaying fast part on top of the slow learner does, by a small margin.
- **Probes of eviction and forgetting**: frozen snapshots of the learner, scored on a never-read control book and on held-out text, show that a book's memory is evicted within a few thousand tokens and that general knowledge is lost outside the stream's domain but not inside it.
- **Long contexts and model size**: on 200k-token books, online learning is worth about half the step from Pythia-14M to Pythia-31M, but Pythia-31M gains as much from it.
- **Update rules across sizes**: plain SGD starves Pythia-70M's MLPs; a per-tensor normalized update recovers part of its cache, Adam produces none at any size.

2 Related work

Dynamic evaluation. Krause et al. [12] adapt a recurrent language model to the recent history of the sequence it is predicting with gradient steps at test time; applied to Transformer-XL the gains are smaller than for weaker models, which the authors take to suggest that Transformers already model re-occurring patterns better than earlier architectures [13]. Rannen-Triki et al. [19] revisit dynamic evaluation for larger models and frame it as memory in weights, a form of context-length extension, with the largest gains under distribution shift. We use the same prequential protocol (score, then update) and ask what the updated weights remember and where.

Cache language models. Kuhn and De Mori [14] mix a cache of recently seen words into an n-gram model; Grave et al. [9] store recent hidden states and their next words and mix the resulting distribution into a neural model. Our unigram cache is the simplest member of this family and the baseline a "cache in the weights" should beat.

Test-time training for language models. Test-time training adapts a model to each test input with a self-supervised loss [22]. For language models, In-Place TTT [6] treats the final projection of the MLP blocks as fast weights updated with a next-token objective; qTTT [2] updates only the query projections on the long context; TTT-E2E [23] meta-learns a model for test-time training on long context. Our finding that MLPs below the output hold the memory is consistent with In-Place TTT's choice of adaptable weights and with the view of feed-forward layers as key-value memories [8].

Fast weights and learned update rules. Fast weights change at inference time to hold short-term memory on top of slow weights [10, 21, 1]; linear transformers can be read as fast weight programmers [20]. Meta-SGD [15] learns a learning rate per parameter. Our slow-plus-fast rule and its meta-learned variant are minimal instances of these ideas applied on top of a pretrained model.

What this work adds. Prior work reports the overall gain of online adaptation. We split it by distance, into local and global parts, by book, against a count cache and by the part of the network that is updated, and we vary the locality of the input and the context length.

3 Setup

Models. Pythia-14M [4] (14.07M parameters: 6 layers of width 128 plus untied input and output embeddings of 6.44M each), Pythia-31M (30.49M) and Pythia-70M (70.43M), all with 6 layers and the same tokenizer; float32 on CPU.

Data. PG-19 test books [18], in the order the Hugging Face dataset gives them; books under 20k tokens (three short stories) are skipped. The main stream is the first 25 books, each cut to its first 20k tokens, concatenated (500k tokens). Two *long* streams use the first test books with at least 100k tokens (5 books cut to 100k) and at least 200k tokens (4 books cut to 200k). Pythia was trained on the Pile [7], whose PG-19 component has 28,602 documents [3], exactly the size of PG-19's training split, so the test books are very likely not in Pythia's training data. The first learning-rate picks use 2 further test books (the *check books*); all later tuning and meta-training use only PG-19 validation books: 10 for picking (*val-pick*), 26 for meta-training and 6 held out.

Protocol. A stream is cut into non-overlapping chunks of $L = 256$ tokens, each scored on its own with no attention to earlier chunks; every token but a chunk's first is predicted from the tokens before it. Online, after scoring a chunk the model takes one plain SGD step (no momentum or weight decay) on the chunk's mean loss: every token is predicted before it is trained on. The frozen model reads the same stream without updates. Some runs update only part of the network (the output embedding, everything else, the MLPs or the attention of all layers, or one layer's MLP). In the *slow and fast* rule the model predicts with $S + F$, where F starts at 0; after each chunk, with g the gradient at $S + F$, $S \leftarrow S - 10^{-4}g$ on all parameters and $F \leftarrow \lambda F - \alpha g$ on the tensors that have a fast part. Without a fast part this reproduces plain SGD at 10^{-4} exactly.

Snapshot probes. To measure what the weights hold at a given point, we copy them (a *snapshot*) and score held-out text with them frozen, then let the learner continue; reading a stream in segments this way gives exactly the same losses as reading it at once. The held-out sets are never trained on: the first 6 PG-19 validation books of the held-out split (tokens 0–3839 of each) and the start of WikiText-103's test split [17] (the articles behind its first 20,480 tokens), detokenized with the WikiText detokenizer of the LM evaluation harness (19,456 tokens).

Locality streams. To vary locality, each of the 25 books is chunked on its own (78 chunks; the remainder is dropped) and the same chunks are read in different orders: the books one after the other ($K = 1$), groups of 5 books with chunks interleaved round-robin inside a group ($K = 5$), all 25 books round-robin ($K = 25$), and one random order (shuffled, seed 0).

Learning rates. Each learning rate is the one with the lowest mean loss on the check books or on val-pick, from a grid of three or four values; from test 6 on, a pick at the edge of its grid extends the grid once by one step. For Pythia-14M with all parameters the pick is 10^{-4} (check-book loss 4.4775, against 4.5474 frozen).

Metrics. The *gain* of a run is the frozen loss minus the run's loss over the same tokens (nats/token); each model is compared with its own frozen loss unless stated. Each scored token is binned by where the previous occurrence of the same token id is: in the *same chunk* (attention can see it), in an *earlier chunk* under 1k, 1k–10k, 10k–100k or at least 100k tokens back, or *never seen*; the 1000 most frequent ids are left out of the bins (139,281 of 498,046 scored tokens remain). Earlier-chunk bins are split by whether that occurrence is in the token's own book or an earlier one. The *locality* is the gain in reading order minus the gain on the same chunks shuffled: the part of the gain that needs the recent material. **Error bars:** all means in the text and tables are token-level; standard errors are computed over books, from the per-book means or, for comparisons of two runs on the same chunks, from the paired per-book differences (books with fewer than 50 tokens in a bin are left out of the SE). A † marks a difference under 2 SE, which we treat as within noise.

4 Results

4.1 The online gain reaches beyond the window and is local

Online learning lowers Pythia-14M's loss from 4.3922 to 4.2929 nats/token (Table 1). For rare tokens last seen in an earlier chunk the gain falls with distance, from +0.301 under 1k tokens back to +0.143 at 1k–10k and is

bin (previous occurrence)	tokens	frozen loss	online, in order	online, shuffled	locality
all tokens	498046	4.3922	$+0.099_{\pm.006}$	$+0.061_{\pm.003}$	$+0.038_{\pm.004}$
same chunk	9881	5.0194	$-0.107_{\pm.014}$	$-0.138_{\pm.009}$	$+0.030_{\pm.015}$
earlier, $<$ 1k	17054	7.3382	$+0.301_{\pm.011}$	$+0.054_{\pm.008}$	$+0.247_{\pm.011}$
earlier, 1k–10k	34076	7.0001	$+0.143_{\pm.009}$	$+0.038_{\pm.005}$	$+0.105_{\pm.006}$
earlier, 10k–100k	44133	7.1181	$+0.010^{\dagger}_{\pm.006}$	$+0.023_{\pm.004}$	$-0.014_{\pm.006}$
earlier, $\geq$ 100k	13178	7.8193	$-0.035_{\pm.009}$	$-0.018_{\pm.008}$	$-0.017_{\pm.007}$
never seen	20959	8.1856	$-0.062_{\pm.013}$	$-0.055_{\pm.011}$	$-0.007^{\dagger}_{\pm.009}$

Table 1: Gain over frozen Pythia-14M (nats/token, ± SE over 25 books) by distance to the token's previous occurrence; online learning on all parameters (lr 10^{-4}), in reading order and with the chunks shuffled. Locality = in order − shuffled. †: under 2 SE.

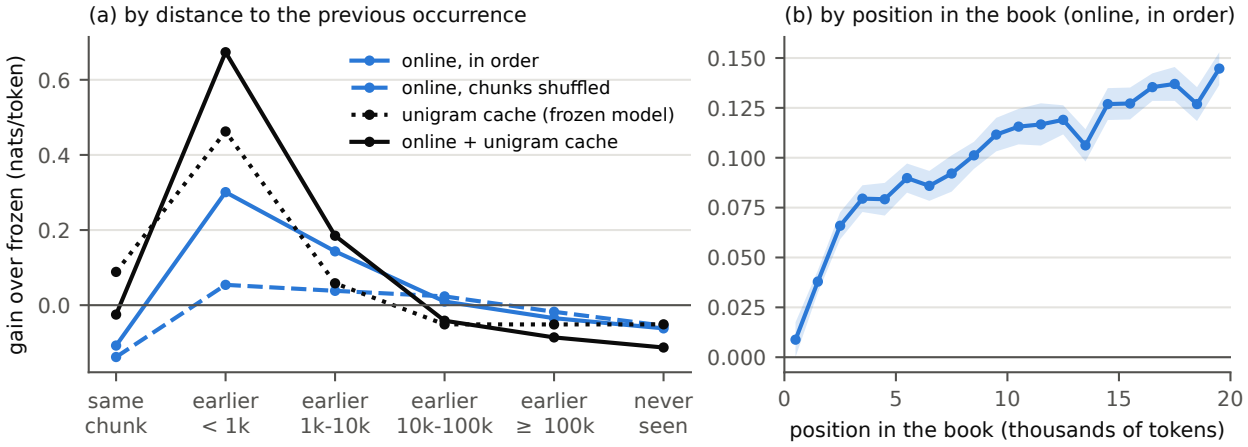


Figure 1: (a) Gain by distance bin for online learning in order and shuffled, the unigram cache on the frozen model, and their combination. (b) Gain of online learning by position in the book, mean over the 24 books that follow a switch, ±1 SE over books.

within noise at 10k–100k; tokens never seen before lose (−0.062), as do tokens repeated inside their own chunk (−0.107), which attention already copies (Figure 1a). After each switch to a new book the gain starts near zero and keeps growing over the 20k tokens of a book (Figure 1b). Shuffling the chunks keeps the training data and removes the order. The shuffled learner keeps 61% of the overall gain, mostly adaptation to PG-19 books in general (it carries over to unseen PG-19 books but not to other text; Section 4.10), but only 18% of the gain under 1k tokens and 27% at 1k–10k: the long-range gain is local (locality +0.247 and +0.105).

4.2 It is mostly about the current book

Table 2 splits the earlier-chunk bins by book. At 1k–10k tokens back, the locality is +0.117 for tokens last seen in the same book and +0.025 for tokens last seen in an earlier book; same-book tokens 10k–20k back still have +0.038, more than closer earlier-book tokens, while earlier-book tokens at 10k–100k have none. The fall of the gain with distance is therefore largely a fall in the share of same-book tokens, although distance still matters within a book. The unigram cache depends more on distance than on the book: it gains on same-book tokens at 1k–10k but already loses at 10k–20k, where online learning still gains +0.065.

4.3 More than a count cache

Table 3 compares online learning with a unigram cache mixed into the frozen model, $p = (1-\lambda)p_{\text{model}} + \lambda p_{\text{cache}}$, with exponentially decaying counts updated after each chunk ($\lambda = 0.05$, half-life 1k tokens). The cache is the better short-range copier (+0.462 under 1k tokens, +0.089 within the chunk), but from 1k tokens on online learning is better (+0.143 against +0.058 at 1k–10k), and overall it gains six times more. The two combine

bin	previous occurrence	tokens	in order	shuffled	locality	unigram cache
earlier, $<$ 1k	same book	16988	$+0.302_{\pm.011}$	$+0.054_{\pm.008}$	$+0.248_{\pm.011}$	$+0.463_{\pm.025}$
earlier, 1k–10k	same book	29680	$+0.155_{\pm.009}$	$+0.038_{\pm.005}$	$+0.117_{\pm.007}$	$+0.069_{\pm.009}$
earlier, 1k–10k	earlier book	4396	$+0.065_{\pm.009}$	$+0.040_{\pm.008}$	$+0.025_{\pm.009}$	$-0.016_{\pm.004}$
earlier, 10k–100k	same book	4814	$+0.065_{\pm.012}$	$+0.027_{\pm.007}$	$+0.038_{\pm.010}$	$-0.050_{\pm.000}$
earlier, 10k–100k	earlier book	39319	$+0.003^{\dagger}_{\pm.006}$	$+0.023_{\pm.004}$	$-0.020_{\pm.006}$	$-0.051_{\pm.000}$

Table 2: Gain over frozen Pythia-14M by whether the previous occurrence is in the same book or an earlier one (± SE over books). The earlier-book part of the $<$ 1k bin (66 tokens) is omitted.

bin	online, all	unigram	online + cache	output, lr 10^{-3}	output, lr 3×10^{-3}	all but output
all tokens	$+0.099_{\pm.006}$	$+0.017_{\pm.003}$	$+0.102_{\pm.007}$	$+0.045_{\pm.003}$	$+0.072_{\pm.005}$	$+0.094_{\pm.005}$
same chunk	$-0.107_{\pm.014}$	$+0.089_{\pm.017}$	$-0.025^{\dagger}_{\pm.020}$	$+0.011^{\dagger}_{\pm.006}$	$+0.092_{\pm.014}$	$-0.105_{\pm.014}$
earlier, $<$ 1k	$+0.301_{\pm.011}$	$+0.462_{\pm.025}$	$+0.673_{\pm.024}$	$+0.061_{\pm.006}$	$+0.251_{\pm.016}$	$+0.290_{\pm.011}$
earlier, 1k–10k	$+0.143_{\pm.009}$	$+0.058_{\pm.008}$	$+0.185_{\pm.012}$	$-0.001^{\dagger}_{\pm.004}$	$+0.066_{\pm.005}$	$+0.139_{\pm.009}$
earlier, 10k–100k	$+0.010^{\dagger}_{\pm.006}$	$-0.051_{\pm.000}$	$-0.041_{\pm.006}$	$-0.039_{\pm.003}$	$-0.054_{\pm.005}$	$+0.009^{\dagger}_{\pm.006}$
earlier, $\geq$ 100k	$-0.035_{\pm.009}$	$-0.051_{\pm.000}$	$-0.086_{\pm.009}$	$-0.075_{\pm.003}$	$-0.146_{\pm.004}$	$-0.034_{\pm.009}$
never seen	$-0.062_{\pm.013}$	$-0.051_{\pm.000}$	$-0.113_{\pm.013}$	$-0.064_{\pm.006}$	$-0.143_{\pm.012}$	$-0.058_{\pm.012}$

Table 3: Gain over frozen Pythia-14M (± SE over books): online learning (all parameters), the unigram cache on the frozen model, both combined, and online learning restricted to the output embedding or to everything else.

within 10k tokens (+0.673 under 1k), but the combination loses beyond, so overall it is no better than online learning alone ($+0.002 \pm 0.002$, within noise). Updating everything but the output embedding reproduces the full learner; updating only the output embedding is a short-range cache at best (+0.251 under 1k at lr 3×10^{-3}, only +0.066 at 1k–10k).

4.4 Eviction: fast, with a long tail

A first test split three books A into halves A1 and A2 of 10k tokens; online learning read A1, then $I \in \{0, 10\text{k}, 100\text{k}\}$ tokens of other books, then A2, and a second run replaced A1 with 10k tokens of a different book B1. The loss on A2 after B1 minus the loss after A1 was +0.038, +0.027 and +0.019 nats/token at $I = 0$, 10k and 100k (mean of 3 books), a slow decline; a unigram cache keeps +0.008 at $I = 0$ and nothing after 10k tokens. But both runs keep learning while they read A2, and this comparison cannot tell whether the A1 run loses A or the B1 run catches up through general adaptation.

Snapshot probes separate the two. For each of 20 books A (paired with a control book C among the 25), the learner reads A1 followed by PG-19 validation books (a different interlude for each A); at each checkpoint, A2 and the same span C2 of the never-read control book are scored frozen with the snapshot (Figure 2). The gain on A2 over the pretrained model is $+0.067 \pm 0.007$ right after A1, halves after about 5k tokens of other books, and is within noise from 30k tokens on ($+0.010 \pm 0.005$). The control never gains (-0.030 ± 0.007 after A1, between -0.011 and -0.032 later), so there is no general adaptation for a comparison run to catch up with: the learner loses A. The *memory of A*, the gain on A2 minus the gain on C2 at the same snapshot, is $+0.097 \pm 0.013$ after A1, halves after about 6k tokens, and keeps a long tail: $+0.044 \pm 0.007$ at 10k, $+0.031 \pm 0.006$ at 100k and $+0.016 \pm 0.008$ at 300k. The first test's slow decline came from scoring A2 online: with frozen probes its own metric is +0.123, +0.040 and +0.021 at $I = 0$, 10k and 100k. The memory is mostly about tokens of A1: rare tokens of A2 that occur in A1 carry $+0.223 \pm 0.029$ at $I = 0$, rare tokens that do not $+0.056 \pm 0.011$. Updating only layer 4's MLP at a higher learning rate (Section 4.7) gives more memory early ($+0.021 \pm 0.003$) but a lower gain on A2 itself (-0.010 ± 0.002): its extra memory is a larger loss on the control book. The slow and fast rule (Section 4.8) adds $+0.023 \pm 0.002$ of memory at $I = 0$, $+0.013 \pm 0.002$ at 10k and nothing measurable from 30k, a half-life of roughly 10k tokens.

4.5 The memory lives in the MLPs

Updating only the MLPs of the six layers (0.79M parameters) keeps most of the long-range gain (+0.216 under 1k tokens, +0.123 at 1k–10k); updating only attention (0.40M) keeps less (+0.142, +0.092). Table 4 goes further,

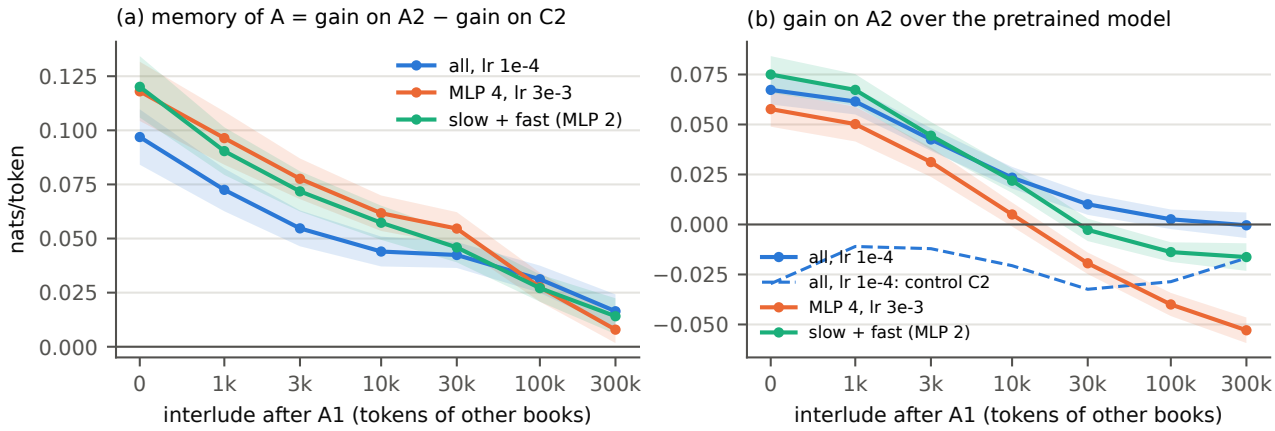


Figure 2: Eviction probed with frozen snapshots (Pythia-14M, mean of 20 books, bands ±1 SE over books). (a) Memory of A: gain on A2 minus gain on a never-read control book C2, scored with the same snapshot, against the number of tokens of other books read after A1. (b) Gain on A2 over the pretrained model, and on C2 (dashed) for plain SGD on all parameters.

bin	all	MLP 1–6	MLP 5–6	L1	L2	L3	L4	L5	L6
lr	10^{-4}	10^{-4}	10^{-3}	10^{-3}	3×10^{-3}	3×10^{-4}	3×10^{-3}	3×10^{-3}	10^{-3}
all tokens	$+0.099_{\pm.006}$	$+0.085_{\pm.005}$	$+0.070_{\pm.006}$	$+0.052_{\pm.004}$	$+0.055_{\pm.007}$	$+0.055_{\pm.004}$	$+0.067_{\pm.006}$	$+0.072_{\pm.006}$	$+0.047_{\pm.004}$
same chunk	$-0.107_{\pm.014}$	$-0.086_{\pm.011}$	$-0.076_{\pm.010}$	$-0.079_{\pm.013}$	$-0.137_{\pm.017}$	$-0.088_{\pm.013}$	$-0.093_{\pm.012}$	$-0.076_{\pm.011}$	$-0.053_{\pm.008}$
earlier, < 1k	$+0.301_{\pm.011}$	$+0.216_{\pm.009}$	$+0.225_{\pm.012}$	$+0.229_{\pm.012}$	$+0.395_{\pm.018}$	$+0.202_{\pm.009}$	$+0.357_{\pm.017}$	$+0.285_{\pm.014}$	$+0.108_{\pm.007}$
1k–10k, same book	$+0.155_{\pm.009}$	$+0.130_{\pm.008}$	$+0.104_{\pm.009}$	$+0.115_{\pm.008}$	$+0.133_{\pm.012}$	$+0.093_{\pm.007}$	$+0.135_{\pm.010}$	$+0.120_{\pm.010}$	$+0.052_{\pm.007}$
10k–100k, same book	$+0.065_{\pm.012}$	$+0.063_{\pm.011}$	$+0.029_{\pm.009}$	$+0.049_{\pm.009}$	$+0.002^{\dagger}_{\pm.015}$	$+0.026_{\pm.010}$	$+0.015^{\dagger}_{\pm.012}$	$+0.028_{\pm.012}$	$+0.009^{\dagger}_{\pm.008}$
never seen	$-0.062_{\pm.013}$	$-0.055_{\pm.011}$	$-0.095_{\pm.015}$	$-0.054_{\pm.011}$	$-0.129_{\pm.015}$	$-0.058_{\pm.011}$	$-0.116_{\pm.013}$	$-0.090_{\pm.015}$	$-0.084_{\pm.012}$

Table 4: Gain over frozen Pythia-14M (± SE over books) when only some MLPs are updated online: all six layers, the last two, or one layer (L1–L6), each at its own learning rate (first row).

updating one layer's MLP (131,712 parameters) at a learning rate picked per layer. Every layer's MLP but the last holds a large part of the memory: the MLP of layer 2 or 4 alone gains +0.395 and +0.357 under 1k tokens, more than the full learner, while the last layer is clearly the weakest (Figure 3a). These single-layer runs need higher learning rates, several at the edge of the grid, and pay with lower overall gains and larger losses on tokens not seen recently (layer 2: +0.055 overall, −0.129 on never-seen tokens); on recent same-book tokens the layer-4 MLP is level with the full learner (difference $+0.008 \pm 0.006$, within noise).

4.6 Locality as a variable

The K streams hold the same chunks in orders of decreasing locality. With plain SGD on all parameters the gain falls steadily as locality decreases: +0.176 on the long books, +0.100 with the books in order ($K = 1$), +0.074 and +0.065 with 5 and 25 books interleaved, and +0.062 shuffled (Table 5, Figure 4). Most of the gain on interleaved streams is the global part that shuffling keeps.

4.7 Making the cache more aggressive does not pay off

A more aggressive cache should gain more when locality is strong and lose more when it is weak, so that settings cross over as locality decreases. They do not (Table 5): updating only layer 4's MLP at a higher learning rate, or with a fast part that decays back to the pretrained weights, is worse than plain SGD on every stream, and the order of the settings never changes. The aggressive settings do gain more on recent same-book tokens (+0.267 and +0.295 against +0.210 on $K = 1$), but they lose far more on everything else (−0.292 and −0.234 against −0.059 on never-seen tokens). The gap to plain SGD narrows as locality grows (for MLP 4 at lr 10^{-2}: 0.113 shuffled, 0.082 with $K = 1$, 0.045 on the long books), so a crossover may exist at stronger locality than we tested. Decay makes weak locality more costly, not less: it throws away the general adaptation that carries most of the

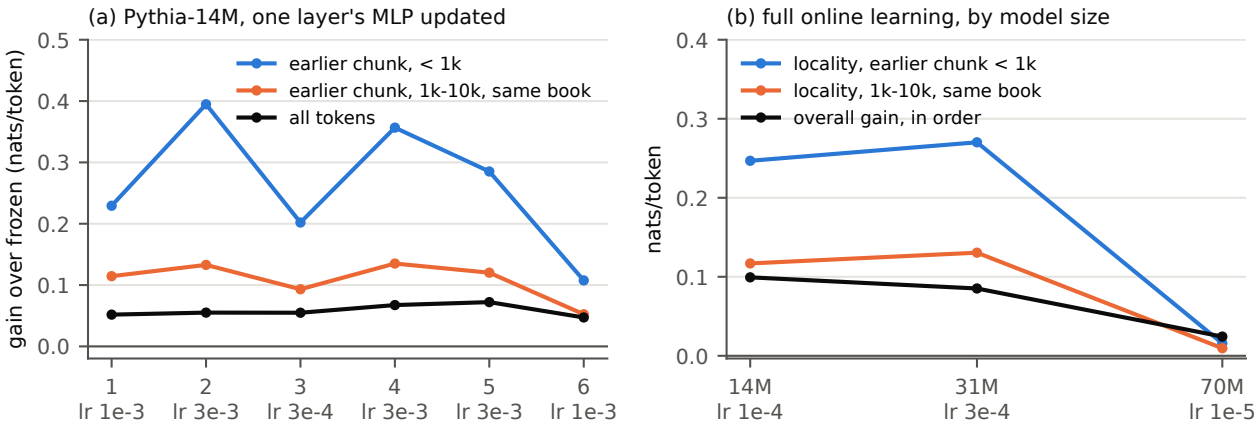


Figure 3: (a) Gain when only one layer's MLP of Pythia-14M is updated online. (b) Locality and overall gain of full online learning for Pythia-14M, 31M and 70M, each at its own learning rate.

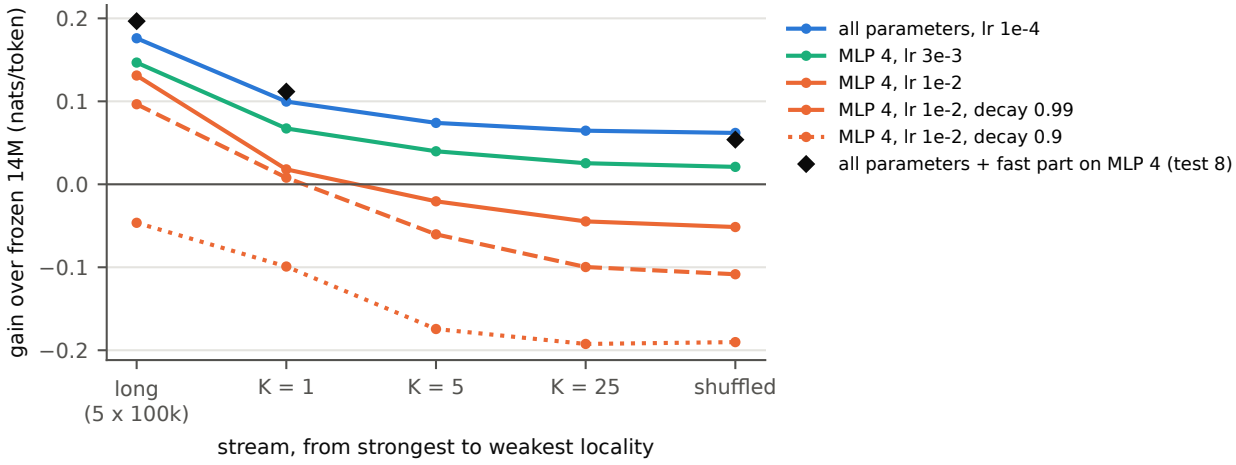


Figure 4: Gain over frozen Pythia-14M against the locality of the stream: plain SGD on all parameters, more aggressive settings on layer 4's MLP, and the slow learner with a fast part on layer 4's MLP (test streams $K = 1$, shuffled and long only).

gain there. Much of the cost is forgetting: these settings lose up to 0.15 nats/token on unseen PG-19 books and up to 0.43 on WikiText (Section 4.10).

4.8 Slow and fast weights

A separate fast part does help, if it sits on top of the slow learner. With the slow part as plain SGD on all parameters and a small fast part ($\alpha = 10^{-3}$, $\lambda = 0.99$, half-life about 18k tokens) on one middle MLP, picked on validation books, the gain over plain SGD is small but robust: $+0.012 \pm 0.001$ with $K = 1$, $+0.021 \pm 0.003$ on the long books and -0.008 ± 0.000 shuffled (Table 6). The extra gain is on recent same-book tokens ($+0.058 \pm 0.004$ on $K = 1$), with almost no extra cost on never-seen tokens. Layers 2 and 4 are equivalent (difference $+0.001 \pm 0.001$ with $K = 1$, within noise); a fast part on all six MLPs hurts on every validation stream, and a smaller α (3×10^{-4}) costs almost nothing shuffled but gains less in order. On the long books the fast part raises the share of the 14M–31M gap closed from 50% to 55%.

setting	long	$K=1$	$K=5$	$K=25$	shuffled	$K=1$: same book <10k	$K=1$: never seen
all parameters, lr 10^{-4}	$+0.176_{\pm.023}$	$+0.100_{\pm.006}$	$+0.074$	$+0.065$	$+0.062_{\pm.003}$	$+0.210_{\pm.009}$	$-0.059_{\pm.013}$
MLP 4, lr 3×10^{-3}	$+0.147_{\pm.023}$	$+0.067_{\pm.006}$	$+0.040$	$+0.025$	$+0.021_{\pm.003}$	$+0.215_{\pm.011}$	$-0.114_{\pm.013}$
MLP 4, lr 10^{-2}	$+0.131_{\pm.030}$	$+0.018^{\dagger}_{\pm.013}$	-0.021	-0.045	$-0.051_{\pm.004}$	$+0.267_{\pm.020}$	$-0.292_{\pm.016}$
MLP 4, decay 0.99	$+0.097_{\pm.023}$	$+0.008^{\dagger}_{\pm.013}$	-0.060	-0.100	$-0.108_{\pm.004}$	$+0.295_{\pm.021}$	$-0.234_{\pm.017}$
MLP 4, decay 0.9	$-0.046_{\pm.019}$	$-0.099_{\pm.011}$	-0.174	-0.192	$-0.190_{\pm.005}$	$+0.138_{\pm.016}$	$-0.191_{\pm.013}$

Table 5: Gain over frozen Pythia-14M (± SE over books) across streams of decreasing locality, and on two token classes of the $K=1$ stream. All but the first setting update only layer 4's MLP; "decay": a fast part on it at lr 10^{-2} that decays back to the pretrained weights. $K=5$ and $K=25$ have no SE (no per-token losses saved).

run	$K=1$	shuffled	long	locality	$K=1$: same book <10k
all parameters, lr 10^{-4} (gain over frozen)	$+0.100_{\pm.006}$	$+0.062_{\pm.003}$	$+0.176_{\pm.023}$	$+0.038_{\pm.004}$	$+0.210_{\pm.009}$
+ fast part, MLP 4 (test 8)	$+0.012_{\pm.001}$	$-0.008_{\pm.000}$	$+0.021_{\pm.003}$	$+0.058_{\pm.005}$	$+0.058_{\pm.004}$
+ fast part, MLP 2 (test 10)	$+0.013_{\pm.002}$	$-0.009_{\pm.000}$	$+0.022_{\pm.003}$		
Meta-SGD lite (test 9)	$+0.012_{\pm.002}$	$-0.020_{\pm.001}$	$+0.020_{\pm.004}$	$+0.070_{\pm.005}$	$+0.079_{\pm.005}$

Table 6: Slow and fast weights on the test streams. First row: plain SGD on all parameters (lr 10^{-4}), gain over frozen Pythia-14M. Other rows: gain over that first row (± SE over books, paired); locality = in order − shuffled. Empty cells were not computed.

4.9 Learning the update rule

We also learned one α and one λ for each of 25 weight matrices (Meta-SGD [15] at the level of tensors), starting from the best hand-picked configuration, with exact second-order meta-gradients through episodes of 16 chunks on 26 validation books. The held-out loss improved by only 0.004 nats/token before stopping, and every learned α stayed within 0.8–1.34 times its starting value. On the test streams the learned rule ties the hand-picked one with $K=1$ and on the long books and loses more shuffled (-0.020 against -0.008): the small fast parts on the other tensors add cost without adding gain.

4.10 General knowledge is lost outside the domain

Snapshots taken every 10k tokens are scored frozen on the held-out sets; *forgetting* is the snapshot's loss minus the pretrained model's, averaged over the snapshots in the second half of the stream (Table 7). On WikiText, Pythia-14M learning online with $K=1$ loses $+0.196 \pm 0.006$ nats/token, 4% of its frozen loss and about twice its gain on the stream; about half of it appears in the first 30k tokens, and over an 800k-token stream it levels off at +0.23–0.27. Raw WikiText, with its tokenization markers, gives +0.312; half of that is on the markers themselves (" @-@ ", spaces before punctuation), a shift in formatting rather than lost knowledge. On unseen PG-19 books there is no forgetting with $K=1$ (-0.006), and the shuffled learner's snapshots are better than the pretrained model (-0.045): the gain that survives shuffling is adaptation to PG-19 books that carries over to unseen ones, at a cost on other text (+0.112 on WikiText). The aggressive settings forget far more, which explains much of their low gains. The fast part forgets slightly more than plain SGD alone ($+0.017 \pm 0.001$ on WikiText) for slightly more gain, so its exchange rate (stream gain divided by WikiText forgetting) is the same (0.53 against 0.51). Pythia-31M at its learning rate forgets more than 14M ($+0.033 \pm 0.005$) for a smaller gain.

4.11 Long contexts and model size

On longer books, online learning is worth a large part of a model-size step. On 4 books of 200k tokens (Table 9, Figure 6), Pythia-14M learning online closes 49% of the gap to a frozen Pythia-31M (51% with the fast part), rising from 7% in the first 10k tokens of a book to at most 79% (84% with the fast part) at 160k–170k, but it stays above frozen 31M at every position. Pythia-31M learning online gains about as much as 14M does ($+0.168 \pm 0.013$ against +0.170 over each model's own frozen loss), stays 0.348 ± 0.013 ahead of 14M learning online, and beats a frozen Pythia-70M in 3 late bins, though not overall (-0.054 ± 0.010).

Online learning does not favor the smaller model. On the main stream the locality is the same at 14M and 31M overall (+0.038 and +0.039; difference $+0.000 \pm 0.001$), and slightly larger at 31M on recent same-book tokens ($+0.018 \pm 0.004$); with plain SGD it almost vanishes at 70M (Table 10), which the next section traces

run	stream	stream gain	WikiText (detok.)	WikiText (raw)	held-out books	gain / forgetting
14M, all, lr 10^{-4}	$K = 1$	$+0.100_{\pm.006}$	$+0.196_{\pm.006}$	$+0.312$	-0.006	0.51
	shuffled	$+0.062_{\pm.003}$	$+0.112_{\pm.004}$	$+0.205$	-0.045	0.55
	long (800k)	$+0.170_{\pm.011}$	$+0.268_{\pm.008}$	$+0.383$	$+0.052$	0.63
14M, MLP 4, lr 3×10^{-3}	$K = 1$	$+0.067_{\pm.006}$	$+0.210_{\pm.006}$	$+0.307$	$+0.041$	0.32
14M, MLP 4, lr 10^{-2}	$K = 1$	$+0.018^{\dagger}_{\pm.013}$	$+0.433_{\pm.009}$	$+0.565$	$+0.142$	0.04
14M, MLP 4, decay	$K = 1$	$+0.008^{\dagger}_{\pm.013}$	$+0.244_{\pm.004}$	$+0.339$	$+0.153$	0.03
14M, slow + fast (MLP 2)	$K = 1$	$+0.113_{\pm.007}$	$+0.213_{\pm.006}$	$+0.333$	$+0.008$	0.53
	shuffled	$+0.053_{\pm.003}$	$+0.132_{\pm.004}$	$+0.224$	-0.035	0.40
31M, all, lr 3×10^{-4}	$K = 1$	$+0.085_{\pm.005}$	$+0.228_{\pm.005}$	$+0.329$	$+0.031$	0.37
	shuffled	$+0.045_{\pm.003}$	$+0.145_{\pm.004}$	$+0.234$	-0.020	0.31

Table 7: Forgetting of held-out text: snapshot loss minus pretrained loss, mean over the snapshots in the second half of the stream (± SE over held-out chunks); positive means knowledge lost. Stream gain over each model's frozen loss (± SE over books). Last column: stream gain / detokenized WikiText forgetting.

size	rule	lr	gain, $K = 1$	locality <1k	locality, all	WikiText forgetting	gain / forg., $K = 1$	shuffled
14M	plain SGD	10^{-4}	$+0.100_{\pm.006}$	$+0.245_{\pm.009}$	$+0.038_{\pm.004}$	$+0.196_{\pm.006}$	0.51	0.55
	Adam	3×10^{-6}	$+0.086_{\pm.005}$	$-0.052_{\pm.009}$	$-0.004^{\dagger}_{\pm.004}$	$+0.140_{\pm.005}$	0.62	0.90
	normalized SGD	10^{-4}	$+0.113_{\pm.008}$	$+0.372_{\pm.011}$	$+0.056_{\pm.006}$	$+0.320_{\pm.008}$	0.35	0.29
31M	plain SGD	3×10^{-4}	$+0.085_{\pm.005}$	$+0.262_{\pm.010}$	$+0.040_{\pm.004}$	$+0.228_{\pm.005}$	0.37	0.31
	Adam	3×10^{-6}	$+0.070_{\pm.004}$	$-0.070_{\pm.011}$	$-0.007_{\pm.003}$	$+0.136_{\pm.004}$	0.52	0.74
	normalized SGD	10^{-4}	$+0.092_{\pm.007}$	$+0.418_{\pm.013}$	$+0.060_{\pm.005}$	$+0.344_{\pm.006}$	0.27	0.13
70M	plain SGD	10^{-5}	$+0.025_{\pm.002}$	$+0.016_{\pm.003}$	$+0.004_{\pm.002}$	$+0.009_{\pm.002}$	2.68	2.60
	Adam	3×10^{-7}	$+0.029_{\pm.002}$	$-0.140_{\pm.010}$	$-0.015_{\pm.002}$	$+0.062_{\pm.003}$	0.46	0.66
	normalized SGD	3×10^{-6}	$+0.057_{\pm.004}$	$+0.084_{\pm.006}$	$+0.018_{\pm.003}$	$+0.039_{\pm.003}$	1.48	1.51

Table 8: Three update rules at three sizes, each at the learning rate picked on val-pick: gain with $K = 1$ over each model's frozen loss, locality (in order − shuffled, ± SE over books, paired), forgetting of detokenized WikiText ($K = 1$, last half of the stream) and the exchange rate (stream gain / WikiText forgetting) with $K = 1$ and shuffled.

partly to the update rule. At these sizes this does not support the hypothesis that smaller models rely more on a cache in their weights.

4.12 Update rules across sizes

With plain SGD, Pythia-70M barely learns (check-book gain 0.011 at its best rate 10^{-5}, against 0.057 for 31M and 0.070 for 14M). Logging the update over the first 40 chunks shows why: at 70M the MLPs move about 100 times less per step than at 14M (relative update $\|\Delta W\|/\|W\|$ of 1.1×10^{-6} against 1.2×10^{-4}), and attention takes 78% of the update. We therefore compare three update rules, each with its learning rate picked on val-pick for every size: plain SGD; Adam [11], with a fresh state for each stream; and normalized SGD, which moves every tensor by the same relative amount per step, $W \leftarrow W - \eta\|W\|g/\|g\|$ (Table 8, Figure 5). Normalized SGD more than doubles 70M's gain (+0.057 against +0.025; $+0.033 \pm 0.002$ paired) and gives it clear locality ($+0.084 \pm 0.006$ within 1k tokens, against +0.016), but far less than at 14M and 31M under the same rule (+0.372 and +0.418). Locality is flat from 14M to 31M and falls at 70M, in absolute terms, relative to the frozen loss (5.1%, 6.1%, 1.3%) and as a share of the gain. At 70M both new rules diverge at the step sizes the smaller models use, and their picks are at the low edge of the grid. Adam works against the cache: in-order runs gain less than shuffled ones at every size (locality −0.05 to −0.14 within 1k tokens), and never-seen tokens lose much more. The smaller model does not pay more general knowledge per nat of gain: the exchange rate is higher at 14M than at 31M under every rule; 70M's is higher still, but at much smaller steps and gains.

5 Discussion and limitations

The results support a qualified version of the cache picture. Online learning gives a small language model a memory beyond its attention window; its long-range part is local, tied mostly to the current book rather than to the recency of single tokens, and stored in the MLPs below the output, consistent with feed-forward layers as

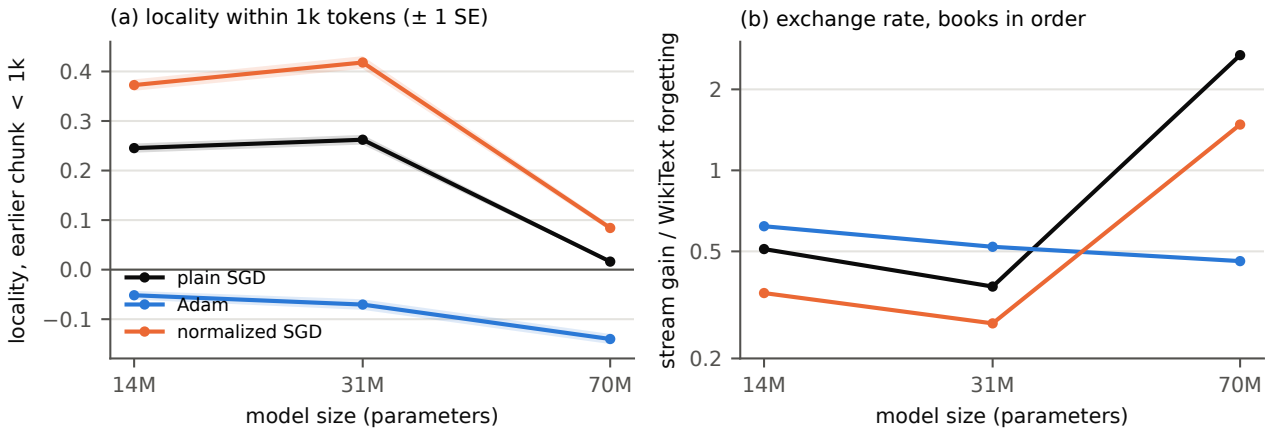


Figure 5: (a) Locality within 1k tokens against model size for three update rules. (b) Stream gain divided by WikiText forgetting ($K = 1$, log scale).

run	loss	gain over frozen 14M	share of 14M–31M gap	seconds
frozen 14M	4.4192			126
14M online, lr 10^{-4}	4.2490	$+0.170_{\pm.011}$	49%	113
14M online + fast part (MLP 2)	4.2387	$+0.180_{\pm.012}$	51%	113
frozen 31M	4.0686	$+0.351_{\pm.010}$	100%	142
31M online, lr 3×10^{-4}	3.9008	$+0.518_{\pm.013}$	148%	182
frozen 70M	3.8465	$+0.573_{\pm.013}$	163%	193

Table 9: Four PG-19 books of 200k tokens. Gain over frozen Pythia-14M (± SE over 4 books) and share of the gap between frozen 14M and frozen 31M closed, over the whole stream; CPU wall time per run.

key-value memories [8] and with MLP weights as the adaptable part in recent test-time training [6]. The memory is short-lived: a book's gain halves within about 5k tokens of other books, with a small tail that lasts beyond 100k. General knowledge is kept within the domain but not outside it: on unseen PG-19 books the learner loses nothing, on WikiText it loses about twice its gain on the stream. Turning the base weights into a more aggressive cache does not pay off at any locality we tested, because the cost on everything else grows much faster than the gain on recent material. A small fast part on top of the slow learner gains more on recent material at little cost on the stream, but its gain is small and it does not reduce the forgetting of general text. The update rule matters as much as the model: plain SGD hardly moves Pythia-70M's MLPs, Adam produces no cache at all, and a per-tensor normalized update gives the largest cache at every size. Context length matters more: over 200k-token books online learning is worth about half the step from 14M to 31M parameters, yet the larger model gains as much, so the cache does not substitute for size at these scales.

These are preliminary results with clear limitations.

- **One seed, few books.** Error bars are across books (25, or 5 and 4 for the long streams), not seeds; with 4–5 books the SE itself is uncertain.
- **Update rule and learning rates.** Most results use plain SGD with per-run learning rates picked from three to five values, several at the edge of their grid; comparisons across layers and sizes mix in the learning rate, and the size of the cache depends on the update rule (Section 4.12).
- **Tuning data.** The first picks, including 31M's learning rate (3×10^{-4}), were made on two test check books, not validation books.
- **Overlapping books.** Two of the four 200k-token books, and all five 100k-token books, are also among the 25 books of the main stream (their first 20k tokens).
- **Missing error bars.** The $K = 5$ and $K = 25$ runs saved no per-token losses and have no error bars.

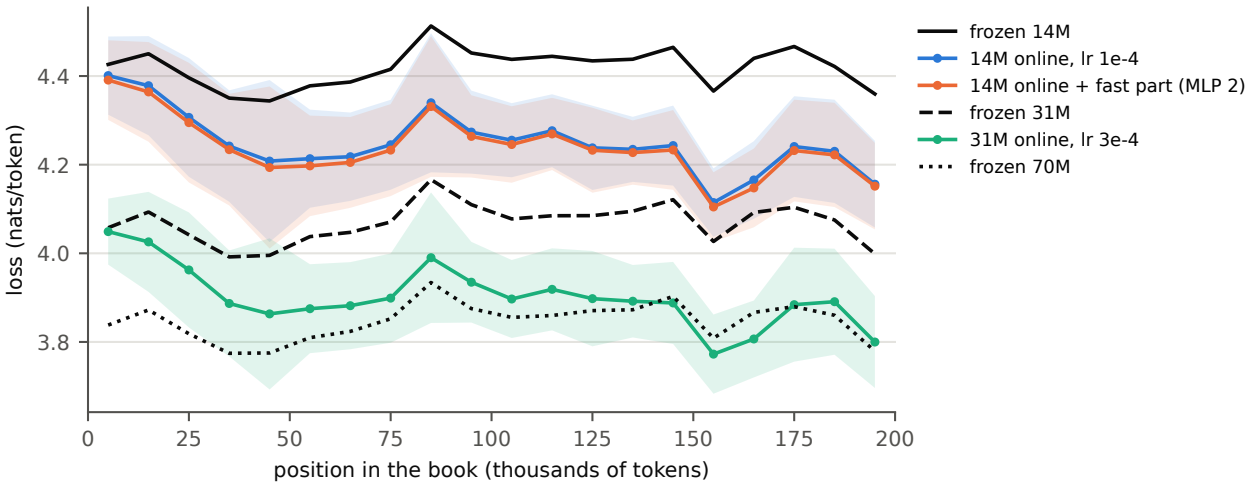


Figure 6: Loss by position in 200k-token books (10k-token bins, mean of 4 books; bands: ±1 SE over books, which mostly reflect differences in book difficulty; paired differences are much tighter).

bin	14M gain	14M locality	31M gain	31M locality	70M gain	70M locality
all tokens	$+0.099_{\pm.006}$	$+0.038_{\pm.004}$	$+0.085_{\pm.005}$	$+0.039_{\pm.004}$	$+0.024_{\pm.002}$	$+0.004_{\pm.002}$
earlier, < 1k	$+0.301_{\pm.011}$	$+0.247_{\pm.011}$	$+0.373_{\pm.015}$	$+0.270_{\pm.009}$	$+0.048_{\pm.004}$	$+0.016_{\pm.003}$
1k–10k, same book	$+0.155_{\pm.009}$	$+0.117_{\pm.007}$	$+0.171_{\pm.009}$	$+0.130_{\pm.006}$	$+0.034_{\pm.004}$	$+0.010_{\pm.003}$
never seen	$-0.062_{\pm.013}$	$-0.007^{\dagger}_{\pm.009}$	$-0.041_{\pm.012}$	$-0.020_{\pm.009}$	$-0.037_{\pm.007}$	$-0.014_{\pm.004}$

Table 10: Gain of full online learning over each model's own frozen loss, and locality, on the main stream (± SE over books). Learning rates: 14M 10^{-4}, 31M 3×10^{-4}, 70M 10^{-5}.

- **Three sizes from one family**; locality is flat from 14M to 31M and falls at 70M, where the rates of the new update rules are at the edge of their grids. Pythia-160M was too slow to run on CPU.
- **Capacity not tested**: we measure how fast one book is evicted, not how many books the weights can hold at once.
- **Held-out text**: forgetting is measured on one encyclopedic text and 6 books; detokenizing WikiText changes it by a third.
- **Wall time** is only a rough compute proxy: frozen runs batch 16 chunks, online runs process one chunk and one update at a time.

Next steps: interleaving many books chunk by chunk to measure how many contexts the weights can hold; more sizes and a finer learning-rate grid with the normalized update; and PolyPythias [24], which provides several pre-training seeds per size, for error bars across models.

6 Conclusion

A small language model that takes one SGD step on each chunk it reads gains a memory beyond its attention window: local, tied mostly to the current book, stored in the MLPs below the output, and more than a count cache beyond 1k tokens. The memory is evicted within a few thousand tokens of other text, and general knowledge outside the domain is lost at about twice the rate the stream is gained. Making that memory more aggressive does not help; a small decaying fast part on one MLP, on top of the slow learner, helps a little and robustly. On long books online learning is worth about half a model-size step, but larger models gain just as much from it, and with a per-tensor normalized update the cache shrinks, but does not vanish, at 70M.

References

[1] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016.

[2] Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Devvrit Khatri, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Sanjay Kale, and Samy Jelassi. Let's (not) just put things in context: Test-time training for long-context LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.13898.

[3] Stella Biderman, Kieran Bicheno, and Leo Gao. Datasheet for the Pile. arXiv:2201.07311, 2022.

[4] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 2397–2430, 2023.

[5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[6] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2604.06169.

[7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027, 2020.

[8] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021.

[9] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations (ICLR)*, 2017.

[10] Geoffrey E. Hinton and David C. Plaut. Using fast weights to deblur old memories. In *Proceedings of the Ninth Annual Conference of the Cognitive Science Society*, pages 177–186, Hillsdale, NJ, 1987. Erlbaum.

[11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[12] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 2766–2775, 2018.

[13] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of transformer language models. arXiv:1904.08378, 2019.

[14] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[15] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. arXiv:1707.09835, 2017.

[16] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[18] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[19] Amal Rannen-Triki, Jorg Bornschein, Razvan Pascanu, Marcus Hutter, András György, Alexandre Galashov, Yee Whye Teh, and Michalis K. Titsias. Revisiting dynamic evaluation: Online adaptation for large language models. arXiv:2403.01518, 2024.

[20] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 9355–9366, 2021.

[21] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[22] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 9229–9248, 2020.

[23] Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context. arXiv:2512.23675, 2025.

[24] Oskar van der Wal, Pietro Lesci, Max Müller-Eberstein, Naomi Saphra, Hailey Schoelkopf, Willem Zuidema, and Stella Biderman. PolyPythias: Stability and outliers across fifty language model pre-training runs. In *International Conference on Learning Representations (ICLR)*, 2025.